

■ LEOPARDO RH

NOTE MÉTHODOLOGIQUE DE CONCEPTION

Méthode hybride de conception, documentation et suivi de projet :
analyse des pratiques, forces et limites

Projet	Leopardo RH — SaaS RH multilingue pour PME africaines et méditerranéennes
Version	v3.3.3 — Mars 2026
Statut	Phase 0 terminée — Prêt pour le codage
Public	Chef de projet, Développeurs, Parties prenantes
Auteur	Équipe conception Leopardo RH

Ce document est à usage interne. Il ne constitue pas un cahier des charges mais une description réflexive et critique de la méthode de travail adoptée.

SOMMAIRE

1.	Introduction et contexte du projet	3
2.	Identification et nommage de la méthode	4
3.	Les piliers de la méthode hybride	5
3.		
1	Documentation comme source de vérité	5
3.		
2	Architecture orientée contrats (API-first)	6
3.		
3	Orchestration multi-agents IA	7
3.		
4	Gestion itérative du backlog	8
3.		
5	Conception UX avant le code	9
4.	Structure du dossier de conception	10
5.	Le système de suivi et d'orchestration	11
6.	Workflow de développement	12
7.	Analyse des forces	13
8.	Analyse des faiblesses et risques	14
9.	Comparaison avec les méthodes établies	15
10.	Recommandations pour la phase de codage	16
11.	Conclusion	17

1. Introduction et contexte du projet

Leopardo RH est une plateforme SaaS RH multilingue conçue pour les PME d'Afrique du Nord et du bassin méditerranéen (Algérie, Maroc, Tunisie, France, Turquie, Sénégal, Côte d'Ivoire). Elle couvre l'intégralité du cycle RH : pointage, gestion des absences, calcul de la paie, suivi des tâches et des évaluations — accessible depuis un navigateur web et une application mobile Android/iOS.

Le projet est développé en monorepo et intègre trois surfaces applicatives distinctes : un backend Laravel 11 avec PostgreSQL, une application Flutter pour les employés, et une interface web Vue.js/Inertia.js pour les gestionnaires. Il supporte quatre langues dont l'arabe en RTL, et s'adapte aux règles locales de droit du travail de chaque pays cible.

Problème résolu

Les PME de ces marchés gèrent encore, en grande majorité, leurs ressources humaines sur Excel, registres papier ou logiciels locaux obsolètes. Cette situation génère en moyenne 3 à 5 heures de corrections par gestionnaire et par mois, avec des risques légaux élevés en cas de contrôle.

Persona	Douleur principale	Coût estimé
DRH PME	Calcul de paie manuel → erreurs → réclamations	4 h/mois
Gestionnaire RH	Suivi absences sur tableur → oublis → conflits	2 h/semaine
Dirigeant	Pas de visibilité temps réel sur les présences	Décisions retardées
Employé	Demander un congé = email + papier + attente	Frustration, turnover

Contexte de conception

La conception a été réalisée entièrement en Phase 0, **avant toute ligne de code**. Elle a produit 31 documents techniques couvrant l'architecture, l'API, la sécurité, la multitenancy, les règles métier, les wireframes, les diagrammes UML, le schéma SQL complet, les modèles Dart Flutter, et les prompts d'exécution pour les agents IA. Au 31 mars 2026, le score de documentation atteint 100 % (31/31 documents validés) et 82+ endpoints API sont documentés.

2. Identification et nommage de la méthode

La méthode utilisée pour concevoir et suivre le projet Leopardo RH ne correspond pas à une approche standard unique. Elle emprunte à plusieurs cadres reconnus tout en les adaptant à un contexte spécifique : équipe réduite, recours intensif aux agents IA comme "développeurs", et couverture documentaire exhaustive en amont du codage. Ce type d'approche porte généralement le nom de **méthode hybride documentée** ou, en gestion de projet logiciel, de **Documentation-Driven Development augmenté par l'IA (DDD-AI)**.

Méthode de référence	Ce qui est emprunté	Ce qui est adapté ou absent
Agile / Scrum	Sprints numérotés, backlog, tableau d'avancement	Pas de daily standup, pas de vélocité d'équipe humaine
API-First Design	Contrats API définis avant le code, OpenAPI 3.0	API doc sert aussi de prompt pour les agents IA
Documentation-Driven Development (DDD)	Toute la documentation produite avant le code	La doc sert d'instruction d'exécution, pas seulement de référence
Orchestration multi-agents	Répartition des tâches par agent IA spécialisé	Pratique émergente — pas de cadre formel existant
UX-First	Wireframes et user flows validés avant le développement	L'appariement se fait avec l'IA, pas avec des utilisateurs réels

Nom retenu pour ce document : Méthode hybride DDD-AI (Documentation-Driven Development assisté par agents IA). C'est une méthode de conception complète en amont, suivie d'une exécution orchestrée entre agents IA spécialisés, pilotée par un chef de projet humain.

3. Les piliers de la méthode hybride

3.1 Documentation comme source de vérité unique

Le principe cardinal de la méthode est que **tout ce qui n'est pas documenté n'existe pas**. Aucun agent IA, aucun développeur ne doit écrire de code sans consulter le dossier de conception correspondant. Cette règle est explicitement inscrite dans le README principal du projet.

Le dossier `docs/dossierdeConception/` contient 31 documents organisés en 20 répertoires thématiques. Chaque document a une version, un statut (Complet / En cours / Obsolète) et un numéro d'ordre. Le fichier `ORCHESTRATION_MAITRE.md` centralise l'état de chacun et sert de tableau de bord global.

Catégorie documentaire	Documents	Rôle
Vision & marché	01_VISION_ET_MARCHE.md	Orienter les décisions produit
API & contrats	02_API_CONTRATS_COMPLET.md openapi.yaml	Source de vérité pour backend et mobile
Architecture & données	03_ERD_COMPLET.md 07_SCHEMA_SQL_COMPLET.sql	Structure de la base de données
Règles métier	05_REGLES_METIER.md 15_GUIDE_AJOUT_PAYS.md	Calculs paie, absences, pointage
Sécurité & accès	07_SECURITE.md 10_RBAC_COMPLET.md 12_SECURITY_SPEC.md	Auth, RBAC 7 rôles, Sanctum
Multitenancy	08_MULTITENANCY_STRATEGY.md 09_TENANT_SERVICE_SPEC.md	Architecture shared/schema PostgreSQL
UX & wireframes	13_USER_FLOWS_VALIDES.md 24_ONBOARDING_GUIDE.md	Flux utilisateurs validés
Mobile Flutter	20_MODELES_DART_COMPLET.md 17 MOCK_DATA_MOBILE.md	Classes Dart + mocks JSON
DevOps & qualité	16_STRATEGIE_TESTS.md 19_CICD_ET_GIT.md	CI/CD GitHub Actions, stratégie de tests
Diagrammes UML	9 fichiers Mermaid (séquences, états, use cases)	Compréhension visuelle des flux

3.2 Architecture orientée contrats (API-First)

Avant d'écrire la moindre ligne de code Laravel, l'équipe a documenté **82+ endpoints API** avec leurs requêtes, réponses, codes HTTP, règles de validation et exemples de payload. Un fichier `openapi.yaml` conforme à la norme OpenAPI 3.0.3 couvre 76 endpoints lisibles par machine.

Cette approche garantit que le backend et le frontend Flutter ne s'écrivent pas en parallèle en faisant des suppositions l'un sur l'autre. Le contrat API est la loi : toute déviation est une régression, pas une évolution spontanée.

- Chaque endpoint documente : méthode HTTP, route, rôles autorisés (RBAC), corps de requête, réponse 200/201/422/401/403/404.
- Les règles de validation Laravel (FormRequest) sont déduites directement des contrats API.
- Les modèles Dart Flutter sont générés depuis les mêmes contrats, garantissant la cohérence de typage.

3.3 Orchestration multi-agents IA

La particularité la plus distinctive de cette méthode est le recours à des **agents IA spécialisés** comme développeurs principaux, chacun confiné à un périmètre technique précis. Un humain joue le rôle de chef de projet et d'orchestrateur.

Agent	Périmètre technique	Prompt(s) associé(s)	Interdit de toucher à
Claude Code	Backend Laravel 11 Vue.js/Inertia	CC-01 à CC-06	Dossier mobile/
Jules (Google)	App Flutter Android + iOS	JU-01 à JU-04	Dossier api/
Cursor	Frontend Vue.js (composants UI)	CU-01	Architecture backend, mobile
v0.dev	Landing page HTML/Tailwind CSS	VO-01	Toute logique métier
Manus	Fichiers de données JSON, YAML, Markdown	MA-01	Tout code source
Claude.ai (chef de projet)	Documentation, revue conception, arbitrage	—	—

Chaque agent reçoit un *prompt master* contenant l'état de la documentation, les fichiers sources de vérité à lire en premier, et les règles de non-chevauchement. L'ORCHESTRATION_MAITRE.md est mis à jour après chaque tâche accomplie.

3.4 Gestion itérative du backlog par sprints nommés

Le projet est découpé en sprints hebdomadaires numérotés (Sprint 0 à Sprint 16) avec des jalons clairs. Chaque tâche porte un identifiant unique (ex. W1-1, S0-7) indiquant la semaine et l'ordre d'exécution.

Phase	Semaines	Contenu principal
Sprint 0	0	Infrastructure, configuration, CI/CD, mocks JSON, dépôts Git
Phase 0 (conception)	Préalable	100 % des documents de conception rédigés et validés
Phase 1 — Infrastructure	1–2	Laravel 11 + PostgreSQL + Redis + Flutter init
Phase 1 — Auth	2–4 (UX d'abord)	Wireframes validés PUIS module d'authentification
Phase 1 — Core	3–4	Pointage, configuration entreprise

Phase 1 — Modules métier	5–8	Absences, avances, tâches, paie, dashboard Vue.js
Phase 2 — Finitions	9–12	Notifications, évaluations, rapports, Super Admin, tests E2E
Phase 3 — Commercialisation	13–16	Landing page, déploiement prod, beta fermée, launch

3.5 Conception UX avant le code (UX-First)

La méthode impose une règle ferme : **aucun endpoint fonctionnel n'est codé avant que l'écran correspondant ne soit validé en wireframe**. Cette règle repose sur une observation pratique : un backend conçu sans UX doit être refondu 2 à 3 fois parce que les champs retournés, l'ordre des étapes et les actions disponibles changent une fois que l'interface est maquettée.

Les wireframes couvrent 6 blocs fonctionnels (A à F) pour 16 écrans principaux, avec des critères de validation explicites pour chaque écran : flux logique, données disponibles dans l'API, support RTL arabe/turc, utilisabilité à une main sur 5,5 pouces.

4. Structure du dossier de conception

Le dossier de conception est organisé selon une numérotation stricte. Chaque répertoire thématique contient un ou plusieurs documents Markdown versionés. La numérotation reflète l'ordre logique de lecture et de dépendance entre les artefacts.

Répertoire	Contenu clé	Utilisé par
00_docs/	CDC et DCT d'origine (PDF/DOCX)	Chef de projet
00_vision_marche/	Vision produit, analyse marché, concurrents	Tous
01_API_CONTRATS/	82+ endpoints — source de vérité API v2.1	Claude Code, Jules
02_personas/	Personas + User Stories	UX, agents
03_modele_eco/	Plans Trial/Starter/Business/Enterprise	Super Admin, billing
04_architecture_erd/	ERD v2.0 + seeders	Claude Code
05_regles_metier/	Calculs paie, pointage, absences, guide pays	Claude Code, tests
07_securite_rbac/	Auth, RBAC 7 rôles, middlewares, webhooks	Claude Code
08_multitenancy/	Stratégie shared/schema, TenantService 7 étapes	Claude Code
09_tests_qualite/	Pyramide de tests, scénarios bloquants	Claude Code, Jules
10_deploiement_cicd/	GitHub Actions, Git workflow, sauvegardes	DevOps
11_ux_wireframes/	User flows, 16 wireframes, onboarding	Jules, Cursor
12_notifications/	Templates push/email, stratégie SSE	Claude Code
13_i18n/	FR/AR/EN/TR + gestion RTL	Tous les agents
14_glossaire/	Dictionnaire technique unifié	Tous
16_MODELES_DART/	9 classes Dart Flutter + enums	Jules
17 MOCK_JSON/	Documentation des mocks JSON mobile	Jules
18_schemas_sql/	Schéma SQL PostgreSQL complet + seeders	Claude Code
19_diagrammes_uml/	9 diagrammes Mermaid (séquences, états...)	Tous
20_templates_pdf/	Bulletin de paie Blade + exports bancaires	Claude Code

La règle de non-modification est stricte : les agents IA ne peuvent pas modifier les documents du dossier de conception sauf via une révision explicitement demandée par le chef de projet et tracée dans le CHANGELOG.md.

5. Le système de suivi et d'orchestration

Le suivi du projet repose sur trois outils documentaires interconnectés, tous maintenus manuellement par le chef de projet après chaque session de travail.

ORCHESTRATION_MAITRE.md — tableau de bord central

Ce fichier unique centralise l'état de tous les modules (Backend, Mobile, Web, Tests) sous forme de tableau avec des indicateurs visuels (■ Non démarré / ■ En cours / ■ Terminé / ■ Bloqué). Il contient également la chronologie semaine par semaine, la répartition des agents, et le journal des décisions techniques.

C'est le premier fichier qu'un agent IA doit lire avant de commencer une tâche. Sa mise à jour est obligatoire après chaque tâche accomplie.

CHANGELOG.md — historique versionné

Le projet suit le format Keep a Changelog avec un versioning sémantique. Chaque version documentée liste précisément les fichiers ajoutés, modifiés et les corrections apportées. Exemple : la version 3.3.3 du 31 mars 2026 détaille l'ajout de 9 diagrammes UML, du fichier openapi.yaml complet et de 5 nouvelles spécifications techniques.

AUDIT_COMPLET_MANQUES.md — gestion proactive des lacunes

Un document d'audit recense en temps réel les manques identifiés dans la documentation et le code. Il classe les lacunes par catégorie (conception, API, wireframes, code, tests) avec un statut (Complet / Partiel / Bloquant). Ce fichier a été le moteur de l'évolution de la documentation de 42 % à 100 % de couverture API entre la v2 et la v3.

Flux de mise à jour

- Le chef de projet initie une session en lisant ORCHESTRATION_MAITRE.md.
- Il assigne une tâche à un agent IA via son prompt master (ex. CC-02 pour le module Auth).
- L'agent IA produit le code ou la documentation demandée.
- Le chef de projet valide, met à jour ORCHESTRATION_MAITRE.md et CHANGELOG.md.
- Si des lacunes sont identifiées, elles sont consignées dans AUDIT_COMPLET_MANQUES.md.

6. Workflow de développement

Le workflow suit la philosophie *documentation d'abord, puis code*, avec une séparation stricte entre les agents et un processus CI/CD automatisé via GitHub Actions.

Stratégie de branches Git

- **main** : branche de production — déploiement automatique sur o2switch via deploy.yml.
- **develop** : intégration continue — tests automatiques via tests.yml à chaque Pull Request.
- **feature/CC-XX** : branches des agents Claude Code (une par module).
- **feature/JU-XX** : branches des agents Jules (Flutter).
- Aucun push direct sur main — tout passe par une PR validée.

CI/CD GitHub Actions

Workflow	Déclencheur	Actions
tests.yml	Pull Request vers develop ou main	Composer install, PHPUnit/Pest, Flutter test, rapport de couverture
deploy.yml	Push sur main	SSH vers o2switch, composer install --no-dev, php artisan migrate, resta

Stratégie de tests (pyramide)

La pyramide de tests est définie avant le code et se décline en trois niveaux, avec des scénarios bloquants explicitement nommés.

Niveau	Outil	Cibles prioritaires	Couverture exigée
Tests unitaires	PHPUnit/Pest	PayrollService, AttendanceService, AbsenceService	100% sur les calculs
Tests de fonctionnalité	Laravel Feature Tests	Auth, TenantIsolation, CRUD Employees	100 % sur TenantIsolationTest
Tests widget/intégration	Flutter Test + Mockito	LoginScreen, AttendanceScreen, AbsenceScreen	100% sur les scénarios principaux

Les tests critiques (TenantIsolationTest, PayrollCalculationTest, AttendanceConflictTest, PlanLimitTest, PublicRegisterTest) sont documentés avec leur code de test attendu dans le dossier de conception — avant d'être écrits.

7. Analyse des forces de la méthode

■ Zéro ambiguïté au démarrage du code

Avec 31 documents complets, 82+ endpoints documentés et un schéma SQL prêt à exécuter, les agents IA démarrent sans devoir faire d'hypothèses. Chaque décision architecturale est tracée dans le journal des décisions techniques (ORCHESTRATION_MAITRE.md §3).

■ Cohérence garantie entre les surfaces

Backend, mobile Flutter et frontend Vue.js utilisent les mêmes contrats API et les mêmes modèles de données. Les modèles Dart sont générés depuis les mêmes spécifications que les FormRequests Laravel, éliminant les désynchronisations de typage classiques dans les projets multi-équipes.

■ Parallélisation réelle des développements

Grâce aux contrats API et aux mocks JSON, l'agent Flutter (Jules) peut développer et tester tous les écrans sans attendre le backend réel. Les mocks JSON couvrent 11 scénarios (login, pointage, absences, paie, notifications) et permettent des tests utilisateurs précoces.

■ Traçabilité complète

Le CHANGELOG versionné (v3.3.3 au 31 mars 2026) enregistre chaque modification avec son contexte et sa justification. L'AUDIT_COMPLET_MANQUES.md garantit qu'aucune lacune n'est ignorée. Toute décision technique est datée.

■ Résilience face au turnover

Un nouveau développeur ou un nouvel agent IA peut être opérationnel en lisant uniquement le README et l'ORCHESTRATION_MAITRE.md. La documentation est auto-suffisante — le projet ne dépend d'aucune connaissance implicite.

■ Scalabilité architecturale planifiée

La multitenancy hybride (shared pour Trial/Starter/Business, schema PostgreSQL pour Enterprise) est documentée avec ses deux implémentations complètes avant d'être codée. La migration d'un tenant shared vers un schéma dédié est spécifiée avec son processus transactionnel en 8 étapes et rollback.

■ Sécurité définie en amont

Le RBAC à 7 rôles, les middlewares de vérification d'abonnement, les limites de plan (PlanLimitMiddleware), la double authentification SuperAdmin/Sanctum et le chiffrement AES-256 des données sensibles sont tous spécifiés avant le premier commit.

■ Couverture multi-pays native

Les règles de calcul de paie, les jours fériés et les formats d'export bancaire pour 7 pays sont documentés dès la Phase 0, permettant une extension géographique sans refactoring architectural.

8. Analyse des faiblesses et risques

■ Coût élevé de la phase de conception

Criticité : Haute

La Phase 0 a produit 31 documents représentant plusieurs semaines de travail conceptuel intense. Pour un projet dont le périmètre évolue rapidement (pivot produit, changement de marché cible), cette investissement initial peut devenir un frein si la documentation ne suit pas l'évolution. **Risque : dérive entre doc et code si les mises à jour ne sont pas disciplinées.**

■ Dépendance aux outils IA externes

Criticité : Moyenne

L'exécution du projet repose sur Claude Code, Jules, Cursor et v0.dev — des services tiers dont les capacités, la disponibilité et les prix peuvent changer. Si l'un de ces agents est discontinué ou modifié significativement, les prompts d'exécution devront être réadaptés. **Risque : obsolescence des prompts ou indisponibilité d'un agent.**

■ Validation UX sans utilisateurs réels

Criticité : Haute

Les wireframes et user flows ont été validés en interne, sans test utilisateur réel sur le terrain. Pour des marchés comme l'Algérie et le Maroc, avec des spécificités culturelles et des niveaux de littératie numérique variables, des surprises sont possibles à la phase de beta. **Risque : ré-conception partielle de l'UX après les premiers tests terrain.**

■ Absence de feedback loop sur les estimations

Criticité : Haute

Les durées estimées (ex. 'Semaine 5-6 : Auth complète') reposent sur des hypothèses sur la vélocité des agents IA, qui n'a pas encore été mesurée sur ce projet spécifique. Aucun historique de sprint n'existe. **Risque : dépassements de planning non anticipés.**

■ Surcharge documentaire potentielle

Criticité : Moyenne

Avec 31 documents + 9 diagrammes UML + un fichier OpenAPI de 136 000 caractères, le risque est que les agents IA ne lisent pas l'intégralité de la documentation avant de coder, produisant des incohérences. **Risque : divergences entre le code produit et les spécifications.**

■ Monorepo : conflits de périmètre entre agents

Criticité : Faible

Bien que les règles de non-chevauchement soient documentées, un monorepo unique avec trois surfaces applicatives augmente le risque de conflits Git et d'interférences entre agents. **Risque : modifications accidentelles d'un périmètre non assigné.**

■ Infrastructure non encore validée

Criticité :
Moyenne

La configuration o2switch (mutualisé) pour héberger Laravel + PostgreSQL + Redis + Supervisor est documentée mais non testée. Les hébergeurs mutualisés imposent des contraintes (accès root limité, versions PHP/PostgreSQL figées) qui peuvent invalider certains choix techniques. **Risque : blocage au moment du déploiement.**

9. Comparaison avec les méthodes établies

Pour permettre au chef de projet de situer précisément cette méthode par rapport aux cadres reconnus du génie logiciel, voici une comparaison structurée sur les critères opérationnels clés.

Critère	Scrum	Waterfall	Kanban	Leopardo DDD-AI
Documentation initiale	Légère (user stories)	Exhaustive avant le code	Minimale	Exhaustive avant le code ■
Livraisons itératives	Sprints 1–4 sem.	Non	Continue (flux)	Sprints semaine + parallélisation
Rôles d'exécution	Équipe humaine scrum team	Équipe humaine spécialisée	Équipe humaine autonome	Agents IA + chef humain
Feedback utilisateur	Sprint review régulière	À la fin du projet	Continu	Beta fermée (semaine 15)
Adaptabilité au changement	Haute (backlog)	Faible (coût élevé)	Haute (flux)	Moyenne (doc à mettre à jour)
Traçabilité décisions	Backlog + retro	Documents formels	Cartes + notes	CHANGELOG + journal décisions ■
Test strategy	TDD possible	Tests en fin	Tests au flux	Pyramide définie avant le code ■
Convient pour équipe IA	Non (prévu humains)	Non	Partiellement	Oui — conçue pour ■

La méthode Leopardo DDD-AI se rapproche le plus du cycle en V (documentation exhaustive en amont) pour la phase de conception, et de Scrum/Kanban pour la phase d'exécution. Son originalité réside dans le remplacement des développeurs humains par des agents IA, ce qui modifie profondément la nature des cérémonies, du feedback et du suivi.

10. Recommandations pour la phase de codage

À l'attention du chef de projet qui prend en main le projet à partir de la Phase 0 terminée, les recommandations suivantes sont issues de l'analyse de la méthode et de ses risques identifiés.

■ Priorité 1 — Valider l'infrastructure avant tout

Avant de lancer CC-01 (init Laravel), vérifier que o2switch supporte bien PostgreSQL 16, Redis et Supervisor dans le plan souscrit. Faire un test de déploiement minimal (Laravel Hello World) pour détecter les contraintes de l'hébergeur au plus tôt.

■ Priorité 2 — Maintenir la discipline de mise à jour documentaire

Nommer une règle explicite : après chaque session de code, mettre à jour ORCHESTRATION_MAITRE.md et CHANGELOG.md AVANT de fermer la session. Une documentation en retard de 3 sessions devient inutilisable.

■ Priorité 3 — Introduire un test utilisateur terrain dès la semaine 8

Ne pas attendre la beta fermée (semaine 15) pour les premiers retours terrain. Identifier 2 à 3 responsables RH dans des PME algériennes ou marocaines et faire valider les wireframes flutter sur leurs propres téléphones.

■ Priorité 4 — Mesurer la vitesse des agents IA dès la semaine 1

Lors du premier sprint d'initialisation, chronométrer le temps réel de CC-01 (init Laravel). Réajuster les estimations des semaines suivantes en conséquence. La planification actuelle est théorique.

■ Priorité 5 — Créer un test d'intégration de régression après chaque module

Après chaque module backend livré (Auth, Pointage, Absences...), écrire et valider un test de régression avant de passer au module suivant. Ne pas laisser s'accumuler les tests non écrits.

■ Priorité 6 — Protéger le périmètre des agents par des hooks Git

Considérer l'ajout d'un hook pre-commit qui vérifie qu'un commit de la branche feature/CC-XX ne modifie pas de fichiers dans mobile/ et vice-versa. La règle documentaire seule ne suffit pas.

11. Conclusion

La méthode de conception adoptée pour Leopardo RH — que ce document nomme **Documentation-Driven Development assisté par agents IA (DDD-AI)** — représente une approche originale et cohérente pour un projet de cette complexité développé par une équipe réduite.

Ses **forces principales** sont la cohérence garantie entre les trois surfaces applicatives, la parallélisation des développements grâce aux mocks, la traçabilité complète des décisions, et la résilience au turnover grâce à une documentation auto-suffisante. Le fait que 100 % de la documentation soit produite et validée avant le premier commit est un atout stratégique majeur.

Ses **faiblesses principales** sont le risque de dérive entre documentation et code si la discipline de mise à jour n'est pas maintenue, l'absence de validation terrain des wireframes, et la dépendance aux outils IA tiers dont la continuité n'est pas garantie.

Le projet entre aujourd'hui dans sa phase la plus critique : le passage de la conception à l'exécution. La qualité de cette transition déterminera si l'investissement documentaire de la Phase 0 se traduit effectivement en gain de temps et de qualité, ou si des ajustements méthodologiques devront être apportés en cours de route.

Message clé pour le chef de projet : Ce dossier est exceptionnel pour ce qu'il contient. Il sera exceptionnel pour ce qu'on en fait. La documentation est prête — à vous de maintenir ce niveau d'exigence pendant les 16 semaines de développement.